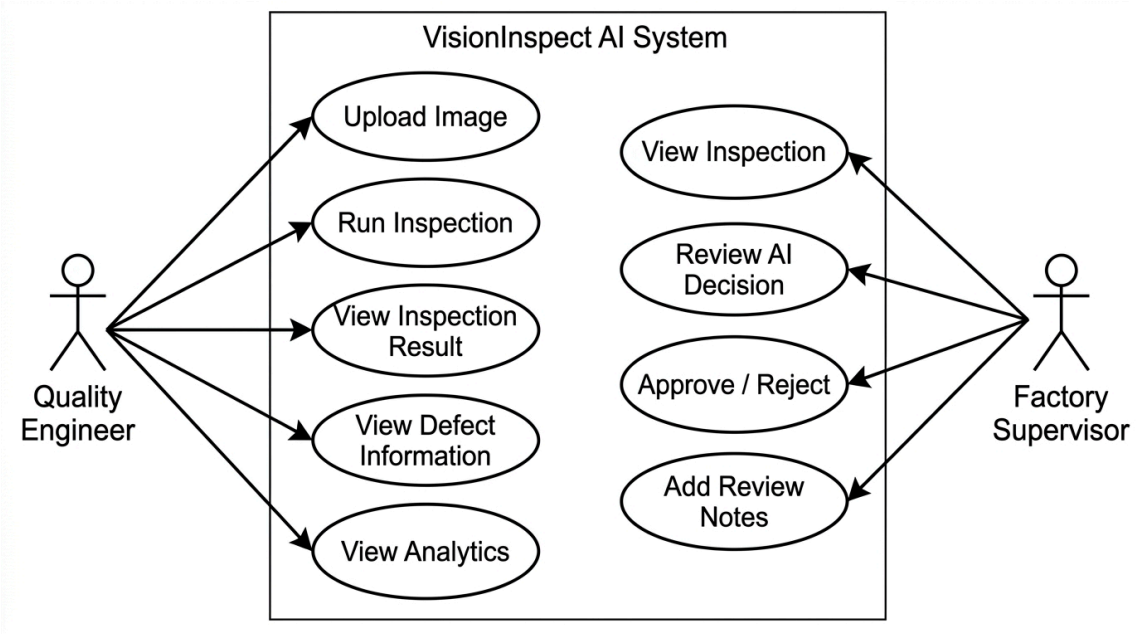


VisionInspect AI — Milestone 2 UML Diagrams

This document contains the minimal, clean, non-technical UML diagrams for Milestone 2 of VisionInspect AI.

1. Use Case Diagram



```
flowchart LR
    QE["Quality Engineer"]
    FS["Factory Supervisor"]

    subgraph System ["VisionInspect AI System"]
        UC1(["Upload Image"])
        UC2(["Run Inspection"])
        UC3(["View Inspection Result"])
        UC4(["View Defect Information"])
        UC5(["View Analytics"])

        UC6(["View Inspection"])
        UC7(["Review AI Decision"])
        UC8(["Approve / Reject"])
        UC9(["Add Review Notes"])
    end

    QE --> UC1
    QE --> UC2
    QE --> UC3
    QE --> UC4
    QE --> UC5

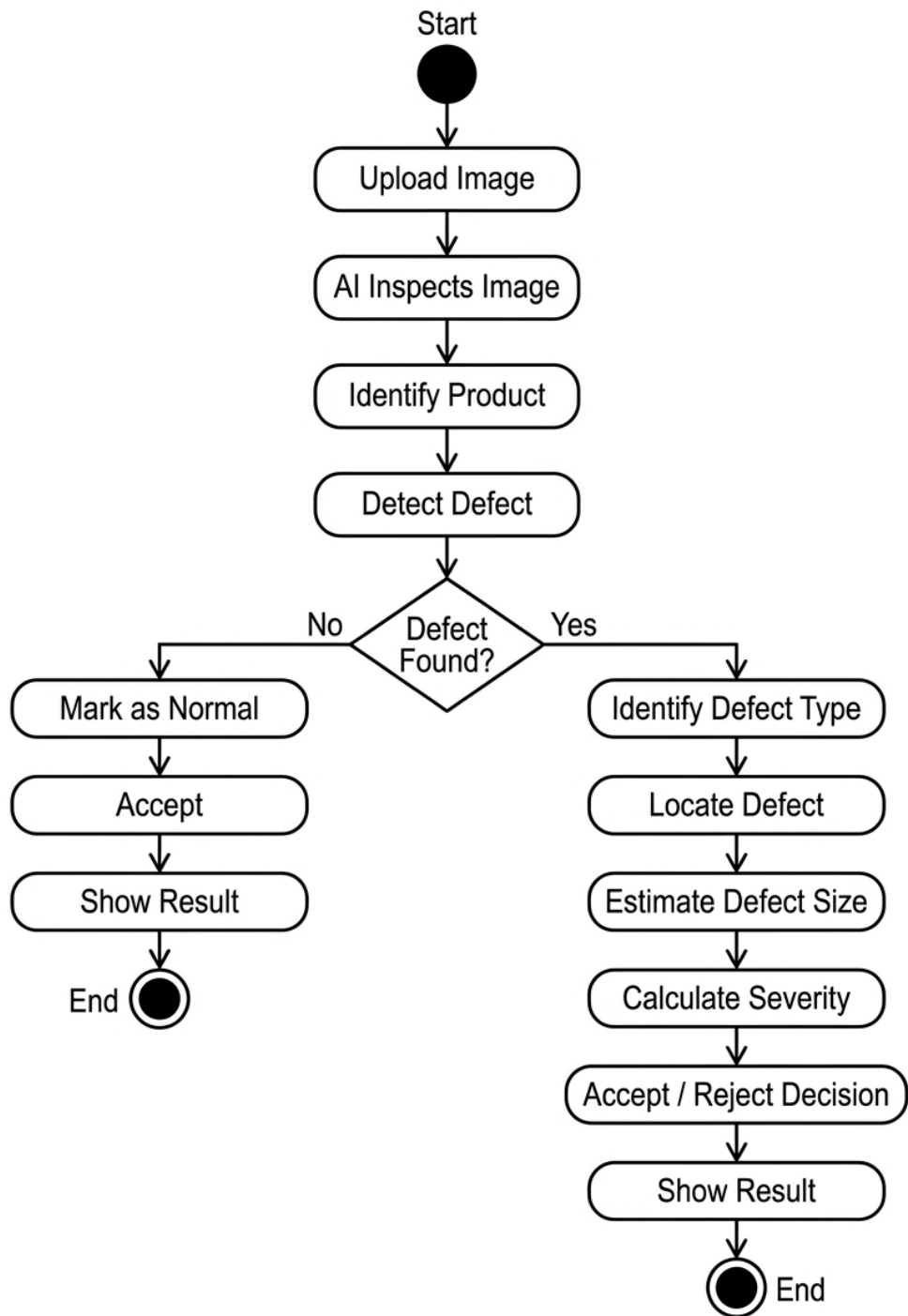
    FS --> UC6
    FS --> UC7
    FS --> UC8
    FS --> UC9
    FS --> UC4
```

FS --> UC6
FS --> UC7
FS --> UC8
FS --> UC9

Explanation:

The Quality Engineer uploads product photos, runs automated inspections, and checks defect details and overall quality trends. The Factory Supervisor audits completed inspections, reviews the AI's recommendations, signs off with an official approval or rejection, and attaches review notes.

2. Activity Diagram — AI Inspection



flowchart TD

```
Start([Start]) --> Upload[Upload Image]
Upload --> Inspect[AI Inspects Image]
Inspect --> Identify[Identify Product]
Identify --> Detect[Detect Defect]
Detect --> Check{Defect Found?}
Check -- No --> Mark[Mark as Normal]
Mark --> Accept[Accept]
Accept --> Show1[Show Result]
Show1 --> End1((End))
Check -- Yes --> IdentifyDef[Identify Defect Type]
IdentifyDef --> Locate[Locate Defect]
Locate --> Estimate[Estimate Defect Size]
Estimate --> Calculate[Calculate Severity]
Calculate --> Decision[Accept / Reject Decision]
Decision --> Show2[Show Result]
Show2 --> End2((End))
```

```

%% Normal Flow
Check -->|No| Normal["Mark as Normal"]
Normal --> Accept["Accept Product"]
Accept --> ShowResult["Show Result"]

%% Defective Flow
Check -->|Yes| Type["Identify Defect Type"]
Type --> Locate["Locate Defect"]
Locate --> Size["Estimate Defect Size"]
Size --> Severity["Calculate Severity"]
Severity --> Decision["Accept / Reject Decision"]
Decision --> ShowResult

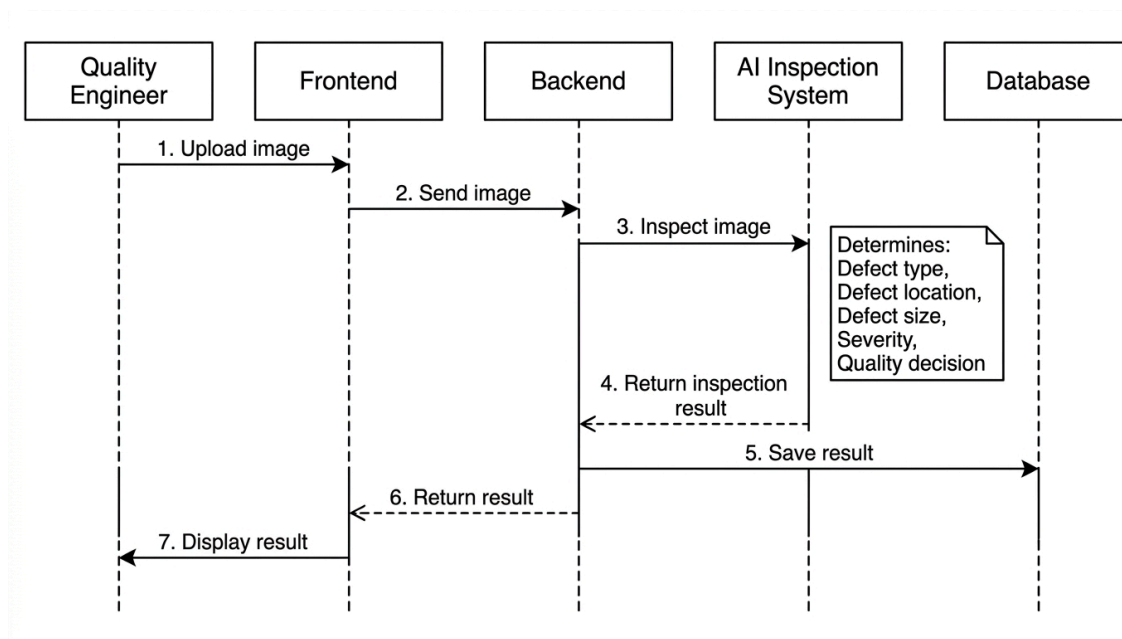
ShowResult --> End([End])

```

Explanation:

When an image is submitted, the system automatically recognizes the product and scans for flaws. If the part is defect-free, it is accepted as normal; if a defect is found, the system identifies its type, pinpoint location, physical size, and severity level before making a final accept-or-reject recommendation.

3. Sequence Diagram — Inspection Flow



```

sequenceDiagram
    autonumber
    actor QE as Quality Engineer
    participant FE as Frontend
    participant BE as Backend
    participant AI as AI Inspection System
    participant DB as Database

```

```

QE-->>FE: Upload image
FE-->>BE: Send image
BE-->>AI: Inspect image

alt Defective Image
    Note over AI: Determines:<br/>1. Defect type<br/>2. Defect location<br/>3.
    Defect size<br/>4. Severity<br/>5. Quality decision
else Normal Image
    Note over AI: Confirms normal product<br/>Decision: Accept
end

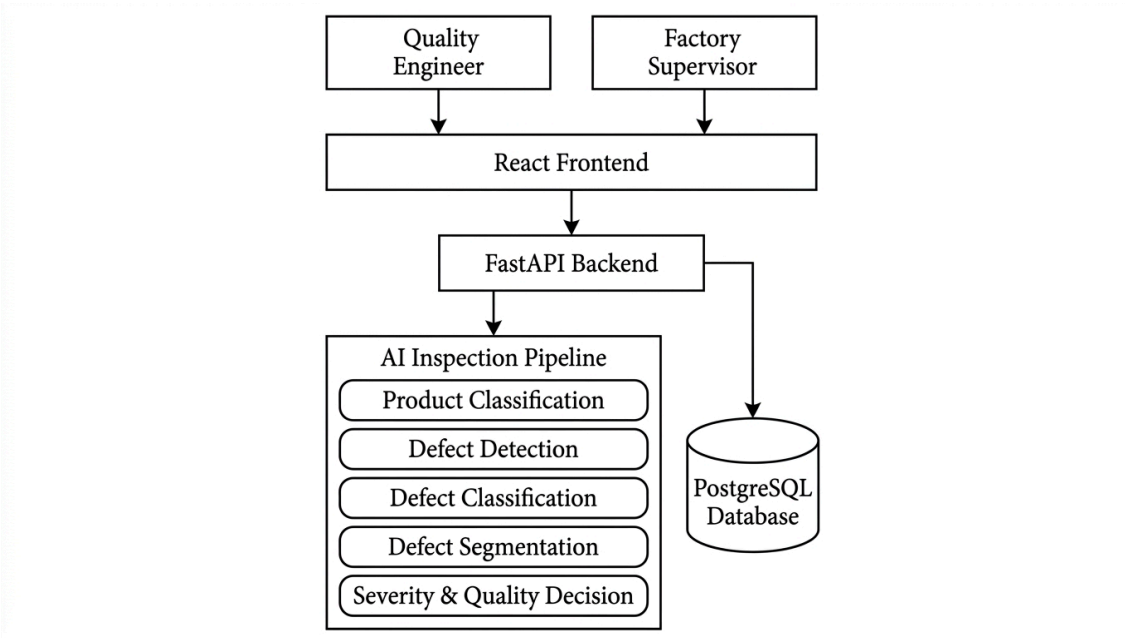
AI-->>BE: Return inspection result
BE-->>DB: Save result
DB-->>BE: Confirm saved
BE-->>FE: Return result
FE-->>QE: Display result

```

Explanation:

The Quality Engineer submits an image through the web dashboard, which forwards it to the server and AI engine for analysis. Once the AI finishes examining the image and measuring any defects, the server stores the results in the database and sends them back to be displayed on screen.

4. High-Level System Architecture Diagram



```

flowchart TD
    subgraph Users ["Users"]
        QE["Quality Engineer"]
        FS["Factory Supervisor"]
    end
    end

```

```
FE["React Frontend"]
BE["FastAPI Backend"]

subgraph Pipeline ["AI Inspection Pipeline"]
    direction TB
    M1["Product Classification"]
    M2["Defect Detection"]
    M3["Defect Classification"]
    M4["Defect Segmentation"]
    M5["Severity & Quality Decision"]

    M1 --> M2
    M2 --> M3
    M3 --> M4
    M4 --> M5
end

DB["PostgreSQL Database"]

QE --> FE
FS --> FE
FE --> BE
BE --> Pipeline
Pipeline --> BE
BE --> DB
```

Explanation:

Factory personnel interact with the system via a React web application connected to a FastAPI backend service. The backend sends incoming images through a 5-step AI pipeline to recognize the product and detect defects, saving all inspection records into a PostgreSQL database.